



Security Audit Report

SMC Computer Supervisor Exam Prep Website

Comprehensive Penetration Test & Vulnerability Assessment

Report Date: August 28, 2026

Classification: CONFIDENTIAL

Prepared for: Authorized Repository Owner

Target: github.com/abhiibabariya-dev/SMC-Computer-Supervisor-Exam-Prep

This report contains sensitive security information. Distribution is restricted to authorized personnel only. Findings and remediation guidance are provided to improve the security posture of the target application.

1. Executive Summary

This security audit was conducted against the SMC Computer Supervisor Exam Preparation website, a static web application hosted on GitHub Pages with Firebase backend services. The assessment followed OWASP testing methodology and manual code review of the client-side application, Firebase security rules, and data handling practices.

Key Statistics

Total Findings: 20

Critical: 3

High: 5

Medium: 8

Low: 4

Most Critical Issues

The three most severe vulnerabilities identified are:

1. VIP Access Bypass Codes Exposed in Client-Side Code (CRITICAL)
Hardcoded access codes allow anyone to unlock premium features without payment.
2. Client-Side Premium Access Control (CRITICAL)
Premium status is enforced only in browser localStorage, not server-side.
3. Insecure Payment Verification (CRITICAL)
Payment verification happens client-side with no server validation.

Overall Risk Rating

HIGH RISK - Immediate Remediation Required

2. Methodology

Testing Approach

The assessment utilized a combination of techniques appropriate for static web applications with cloud backend services:

- Manual source code review of all HTML, JavaScript, and configuration files
- Firebase security rules analysis and access control review
- Client-side data flow and storage analysis
- Authentication and authorization flow mapping
- Payment processing and premium access control review
- Privacy and data protection assessment

Frameworks Referenced

- OWASP Top 10 (2021) - Web Application Security Risks
- OWASP ASVS (Application Security Verification Standard)
- OWASP Testing Guide
- Firebase Security Best Practices
- GDPR / Privacy-by-Design Principles

Scope Limitations

This assessment was a static analysis (white-box) review of the application source code and configuration. Dynamic testing against the live environment was not performed as it would require authenticated access and could impact production services. Findings are based on code review and security architecture analysis.

3. Detailed Findings

SMC-001: VIP Access Bypass Codes Exposed in Client-Side Code

CRITICAL

OWASP: A05:2021 Security Misconfiguration / A01:2021 Broken Access Control

Affected Files:

vip-access.html (lines 80-87), premium.html (lines 650-657), mock-test.html (lines 541-549)

Description:

The application contains hardcoded VIP access codes that unlock premium features without payment. These codes are obfuscated using character code arrays but are trivially reversible by anyone with access to the source code. Three VIP codes are defined in vip-access.html: "abhi2611vip", "bhai2611vip", and "behen2611vip". Additionally, premium.html contains three more codes: "SMC-OWNER-2026", "SMC-FAMILY-BRO", and "SMC-FAMILY-SIS".

Risk & Impact:

Anyone can extract these codes from the client-side JavaScript, decode them, and gain free premium access. This completely undermines the paid subscription model and represents a direct revenue loss. The obfuscation provides no real security as the decoding logic is visible in the same file.

Proof of Concept:

1. Open vip-access.html in a browser
2. View source / inspect element
3. Extract the `_codes` array (lines 81-85)
4. Decode: `[97,98,104,105,50,54,49,49,118,105,112] = "abhi2611vip"`
5. Enter "abhi2611vip" in the access code field
6. Premium access is granted via `localStorage.setItem("smc_vip", "true")`

Remediation:

1. Remove all client-side VIP codes immediately
2. Move premium entitlement verification to Firebase server-side rules
3. Premium status should be stored in Firebase user profile, verified on every request
4. Use Firebase Custom Claims for admin/premium users instead of client-side flags
5. Implement server-side payment verification through Cloud Functions

SMC-002: Client-Side Premium Access Control

CRITICAL

OWASP: A01:2021 Broken Access Control

Affected Files:

premium.html (lines 680-697, 864-941), mock-test.html (device lock verification)

Description:

Premium access is controlled entirely by client-side JavaScript checks against `localStorage`. The application checks `localStorage.getItem("smc_premium_key")` and `localStorage.getItem("smc_vip")` to determine premium status. The `"DeviceLock.verify()"` function (premium.html lines 565-646) only validates a locally-stored device fingerprint hash, which can be forged or copied.

Risk & Impact:

An attacker can simply set `localStorage` values to gain premium access without any payment. The device fingerprinting is cosmetic security theater - the fingerprint is generated client-side and stored locally, so it provides no real protection against tampering.

Proof of Concept:

1. Open the website in browser
2. Open Developer Console
3. Execute: `localStorage.setItem("smc_vip", "true")`
4. Navigate to premium content
5. Access is granted

Remediation:

1. Store premium entitlement in Firebase user profile (server-side)
2. Verify entitlement on every page load via Firebase rules
3. Use Firebase Custom Claims for premium status

4. Remove all localStorage-based premium checks
5. Enforce access control in Firebase security rules, not client code

SMC-003: Insecure Payment Verification (Client-Side)

CRITICAL

OWASP: A04:2021 Insecure Design / A08:2021 Software and Data Integrity Failures

Affected Files:

premium.html (lines 864-941), subscription-payments.js, premium-flow.js

Description:

Payment verification occurs entirely client-side. The `verifyPayment()` function generates an "access key" by hashing `orderNumber + txnId + deviceFP` - all values the client controls. There is no server-side validation that a UPI payment actually occurred. The Firebase `payment_requests` collection accepts submissions from any authenticated user without verifying payment authenticity.

Risk & Impact:

Attackers can submit fake payment requests and self-approve premium access. The access key generation algorithm is visible in source code, allowing key forgery. No actual UPI payment verification occurs.

Proof of Concept:

1. Complete payment flow with any UTR number
2. Access key generated: `"SMC-" + simpleHash(orderNumber + txnId + deviceFP)`
3. `simpleHash` is FNV-1a (visible in source, lines 943-947)
4. Attacker can forge valid keys for any order/transaction combination

Remediation:

1. Implement server-side payment verification via Firebase Cloud Functions
2. Integrate with UPI payment gateway webhook verification
3. Use Firebase Cloud Function to validate payment before granting entitlement
4. Store approved payments with server-generated signatures
5. Never trust client-submitted payment status

SMC-004: Firebase API Key Exposed in Client-Side Code

HIGH

OWASP: A05:2021 Security Misconfiguration

Affected Files:

firebase-config.js (lines 9-18)

Description:

The Firebase Web API key (`AlzaSyBsk3fKL8bmGZM8OY3g7mtLbAym0V5Slc0`) is exposed in client-side JavaScript. While Firebase web API keys are designed to be public, they **MUST** be protected by strict security rules. The current rules have vulnerabilities (see SMC-006) that make this exposure dangerous.

Risk & Impact:

With the API key exposed and weak security rules, attackers can potentially read/write Firebase data directly via REST API. Combined with SMC-006, this allows unauthorized data access.

Proof of Concept:

1. Extract API key from `firebase-config.js`
2. Use Firebase REST API: `GET https://[project].firebase.database.app/users.json?auth=`
3. If rules are misconfigured, retrieve all user data

Remediation:

1. Configure Firebase API key restrictions in Google Cloud Console
2. Restrict key to specific APIs (Identity Toolkit, Firebase Realtime Database)
3. Add HTTP referrer restrictions
4. Ensure security rules are deny-by-default with explicit grants
5. Regularly audit API key usage

SMC-005: Broken Access Control - Admin Detection

HIGH

OWASP: A01:2021 Broken Access Control

Affected Files:

admin-console.html (lines 12, 18-20), login.html (line 30)

Description:

Admin privileges are determined by client-side email comparison: `isAdmin(u)` returns `u.email.toLowerCase() === "abhibabariya007@gmail.com"`. While Firebase security rules also enforce admin checks, the client-side redirect is the first line of defense. If an attacker can bypass the client check (e.g., via modified JavaScript), they may access admin functionality.

Risk & Impact:

Client-side admin detection is not a security control - it only affects UX. The real protection is Firebase rules, but any weakness there (SMC-006) becomes critical. Admin email is also hardcoded and visible to all users.

Proof of Concept:

1. Admin email visible in source: `abhibabariya007@gmail.com`
2. Client-side check can be bypassed by modifying JavaScript
3. Firebase rules are the only real protection

Remediation:

1. Use Firebase Custom Claims for admin role (not email comparison)
2. Enforce admin checks in Firebase security rules (server-side)
3. Never rely on client-side role detection for security
4. Remove hardcoded admin email from client code

SMC-006: Firebase Security Rules Allow Authenticated User Writes to PII Collections

HIGH

OWASP: A01:2021 Broken Access Control

Affected Files:

firebase-rules.json (lines 48-89)

Description:

The security rules allow any authenticated user to write to `audit`, `auth_audit`, `visits`, `clicks`, `leads`, and `security` collections. While reads are restricted to admin, writes from any authenticated user create data integrity and injection risks. For example, the "leads" collection (lines 76-82) accepts name/email from any user without validation beyond `hasChildren` check.

Risk & Impact:

Malicious authenticated users can inject fake tracking data, spam collections, or potentially exploit weak validation. The "visits" collection stores IP, geo-location, and device data from any user without rate limiting.

Proof of Concept:

1. Authenticate with any valid account
2. POST to `/visits.json` with arbitrary data
3. Data is accepted (rules line 62-68 allow any authenticated write)
4. Repeat to spam/inject data

Remediation:

1. Restrict writes to server-side only (Cloud Functions)
2. Add validation rules for all fields
3. Implement rate limiting via Cloud Functions
4. Use Firebase App Check to prevent unauthorized API access
5. Remove client-side writes to analytics collections

SMC-007: PII Collection Without Encryption or Consent

HIGH

OWASP: A02:2021 Cryptographic Failures / GDPR Violation

Affected Files:

tracker.js (lines 4-26, 68-87), firebase-rules.json (audit/security/visits collections)

Description:

The tracker.js collects extensive PII including IP address, geo-location (city, region, country, ISP), device fingerprint, screen resolution, hardware specs, and browsing behavior. This data is stored in Firebase without encryption and without explicit user consent. The GeoIP lookup (ipwho.is) reveals precise location data.

Risk & Impact:

Mass collection of PII without consent violates GDPR/privacy regulations. Data breach would expose sensitive user information. No data retention policy is enforced.

Proof of Concept:

1. Visit any page with tracker.js loaded
2. IP, location, device data collected automatically
3. Data written to Firebase /visits without consent
4. No opt-out mechanism exists

Remediation:

1. Implement cookie consent banner with granular controls
2. Minimize PII collection to essential data only
3. Encrypt PII at rest in Firebase
4. Add data retention and deletion policies
5. Document data processing in Privacy Policy (currently inadequate)

SMC-008: No Content Security Policy (CSP) Implemented

MEDIUM

OWASP: A05:2021 Security Misconfiguration

Affected Files:

All HTML files (no CSP meta tags or headers)

Description:

None of the HTML pages include a Content Security Policy. External scripts are loaded from multiple CDNs (gstatic.com, jsdelivr.net, cdn.jsdelivr.net, googleapis.com) without integrity attributes (SRI). This creates XSS risk if any CDN is compromised.

Risk & Impact:

Without CSP, successful XSS attacks can execute arbitrary JavaScript, steal Firebase tokens, access localStorage premium keys, and exfiltrate user data.

Proof of Concept:

1. If any CDN script is compromised, malicious code executes in user context
2. No CSP to block inline scripts or unauthorized sources
3. No SRI hashes to detect modified scripts

Remediation:

1. Add CSP meta tags or server headers to all pages
2. Use strict CSP: default-src self, script-src with nonces
3. Add SRI integrity attributes to all external scripts
4. Enable X-XSS-Protection and other security headers

SMC-009: Sensitive Data in localStorage

MEDIUM

OWASP: A04:2021 Insecure Design

Affected Files:

premium.html (lines 533, 569-583), gate.js (line 21), vip-access.html (lines 122-123)

Description:

Premium keys, VIP status, device bindings, and user profiles are stored in localStorage. This data persists across sessions and is accessible to any JavaScript running on the page, including injected scripts (XSS). The device fingerprint binding (premium.html lines 521-562) stores access keys in three locations: localStorage, IndexedDB, and cookies.

Risk & Impact:

XSS attacks can steal premium keys and session data. localStorage is accessible to all scripts on the origin. No encryption or protection for sensitive values.

Proof of Concept:

1. XSS vulnerability (if present) can read localStorage
2. localStorage.getItem("smc_premium_key") exposes premium key
3. Data persists indefinitely, increasing exposure window

Remediation:

1. Store premium entitlements server-side (Firebase)
2. Use httpOnly cookies for session tokens where possible
3. Encrypt sensitive localStorage values
4. Implement automatic key rotation/expiry

SMC-010: Client-Side Form Validation Only

MEDIUM

OWASP: A04:2021 Insecure Design

Affected Files:

login.html (form validation), premium.html (goStep validation), all forms

Description:

All form validation is performed client-side. While Firebase security rules provide some server-side validation, the application relies heavily on client checks for data integrity. Account enumeration is possible via distinct error messages for invalid email vs. wrong password.

Risk & Impact:

Client-side validation can be bypassed. Account enumeration enables targeted attacks. Data integrity depends on Firebase rules which have gaps (SMC-006).

Proof of Concept:

1. Intercept signup request, modify validation-bypassed data
2. Firebase rules may not catch all invalid inputs
3. Error messages distinguish "user not found" from "wrong password"

Remediation:

1. Implement server-side validation in Firebase rules
2. Use generic error messages for auth failures
3. Add Cloud Functions for critical data validation
4. Implement account enumeration protection

SMC-011: Question Banks Exposed Client-Side

MEDIUM

OWASP: A01:2021 Broken Access Control (Business Logic)

Affected Files:

mcq-bank.js (lines 1-361), mock-test.html (embedded questions), exam.html

Description:

All examination questions AND their answers are embedded in client-side JavaScript. The mcq-bank.js file contains 100+ questions with correct answer indices (q.a property). Mock tests in mock-test.html include complete question banks with explanations.

Risk & Impact:

Paid premium content (answers to mock tests) is freely extractable from source code. Anyone can scrape answers without purchasing premium access. Undermines the value proposition.

Proof of Concept:

1. Open mcq-bank.js in browser
2. Parse questions array with answer indices
3. All answers accessible without payment

Remediation:

1. Serve questions from Firebase (protected by rules)
2. Separate questions from answers server-side
3. Use server-side quiz logic for premium content
4. Implement answer reveal only after submission

SMC-012: No Session Timeout on Admin Console

MEDIUM

OWASP: A07:2021 Identification and Authentication Failures

Affected Files:

login.html (line 84: setPersistence LOCAL), admin-console.html

Description:

Firebase Auth persistence is set to LOCAL (login.html line 84), meaning admin sessions persist indefinitely across browser restarts. The admin console has no session timeout or re-authentication requirement for sensitive operations.

Risk & Impact:

Admin sessions remain valid indefinitely. If an admin device is compromised or left unattended, attackers gain persistent admin access.

Proof of Concept:

1. Admin logs in once
2. Session persists across browser restarts (LOCAL persistence)
3. No timeout forces re-authentication

Remediation:

1. Use SESSION persistence for admin accounts
2. Implement idle timeout (15-30 min)
3. Require re-authentication for sensitive operations
4. Add "active session" monitoring

SMC-013: UPI ID Exposed via Client-Side Decoding

LOW

OWASP: A05:2021 Security Misconfiguration (Information Disclosure)

Affected Files:

premium.html (lines 411-415, 650-657)

Description:

The UPI payment ID is encoded as character codes and decoded at runtime (premium.html lines 411-415). While this is public payment information, exposing it in client code allows scraping and potential targeted attacks.

Risk & Impact:

UPI ID visible to anyone inspecting source. Low risk but unnecessary exposure.

Proof of Concept:

Not provided

Remediation:

1. Load UPI ID from Firebase (protected config)
2. Or accept as unavoidable but document in privacy policy

SMC-014: EmailJS Public Key Exposed**LOW**

OWASP: A05:2021 Security Misconfiguration (Information Disclosure)

Affected Files:

receipt.js (line 8), premium.html (lines 1027-1033)

Description:

EmailJS public key (R9-pkw_CjpX-5Zayl) is exposed in client code. This is normal for EmailJS but combined with template injection risks, it could be abused for spam.

Risk & Impact:

Public key exposure is standard for EmailJS. Risk is low but templates should be secured.

Proof of Concept:

Not provided

Remediation:

1. Use EmailJS domain allowlist
2. Rate-limit email sends via Cloud Functions
3. Monitor EmailJS usage for abuse

SMC-015: Device Fingerprinting Without Consent**MEDIUM**

OWASP: GDPR / Privacy Violation

Affected Files:

premium.html (lines 420-647: DeviceLock engine)

Description:

The application performs extensive device fingerprinting (canvas, WebGL, audio context, fonts, screen, hardware) without user consent. This creates a persistent tracking identifier that survives browser cleanup in some cases (IndexedDB, cookies).

Risk & Impact:

Privacy violation - tracking users without consent. Potential GDPR non-compliance. Fingerprint can be used for surveillance.

Proof of Concept:

Not provided

Remediation:

1. Obtain explicit consent before fingerprinting
2. Document fingerprinting in Privacy Policy
3. Provide opt-out mechanism
4. Limit fingerprint persistence

SMC-016: No Rate Limiting on Authentication**MEDIUM**

OWASP: A07:2021 Identification and Authentication Failures

Affected Files:

login.html (Firebase Auth), firebase-rules.json

Description:

While Firebase Auth has built-in protection, the application has no custom rate limiting for login attempts or payment submissions. Account enumeration via error messages is possible.

Risk & Impact:

Brute force and credential stuffing attacks possible. Account enumeration enables targeted attacks.

Proof of Concept:

Not provided

Remediation:

1. Implement rate limiting via Cloud Functions
2. Add CAPTCHA for repeated failures
3. Use generic auth error messages
4. Monitor for suspicious login patterns

SMC-017: Insecure Redirects and External Script Loading**LOW**

OWASP: A05:2021 Security Misconfiguration

Affected Files:

All HTML files (external CDN scripts), premium.html (redirects)

Description:

Multiple external scripts loaded without SRI integrity checks. Redirects use location.replace with user-controlled return URLs (sessionStorage smc_auth_return).

Risk & Impact:

CDN compromise could lead to XSS. Open redirect possible if return URL not validated.

Proof of Concept:

Not provided

Remediation:

1. Add SRI hashes to all external scripts
2. Validate return URLs against allowlist
3. Use relative URLs where possible

SMC-018: Admin Email Case Sensitivity Issues**LOW**

OWASP: A04:2021 Insecure Design

Affected Files:

firebase-rules.json (line 7), login.html (line 30), admin-console.html (line 1)

Description:

Firebase security rules compare admin email with lower() function, but client-side checks use .toLowerCase() on user input. If Firebase Auth stores email in different case than expected, admin access may fail. Error message in admin-console.html acknowledges this issue.

Risk & Impact:

Potential admin lockout if email case mismatch. Operational issue, not direct security risk.

Proof of Concept:

Not provided

Remediation:

1. Use Firebase Custom Claims instead of email comparison
2. Normalize email case consistently
3. Document admin onboarding procedure

SMC-019: Automated Reconnaissance Facilitation**LOW**

OWASP: A05:2021 Security Misconfiguration (Information Disclosure)

Affected Files:

robots.txt, sitemap.xml, .nojekyll

Description:

robots.txt and sitemap.xml expose complete site structure. While not a vulnerability itself, it facilitates automated reconnaissance of admin pages and sensitive endpoints.

Risk & Impact:

Attackers can easily discover admin-console.html, vip-access.html, and other sensitive pages.

Proof of Concept:

Not provided

Remediation:

1. Exclude admin/sensitive pages from sitemap
2. Add noindex to admin pages (already done)
3. Use obscure URLs or additional auth for admin

SMC-020: Missing Security Headers**MEDIUM**

OWASP: A05:2021 Security Misconfiguration

Affected Files:

All HTML files (no security headers)

Description:

The application does not implement security headers: X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy, or HSTS. GitHub Pages has limited header control but meta tags could be added.

Risk & Impact:

Missing headers enable clickjacking, MIME sniffing, and information leakage via referrers.

Proof of Concept:

Not provided

Remediation:

1. Add meta tags for X-Content-Type-Options, Referrer-Policy
2. Use CSP meta tag (see SMC-008)
3. Consider custom domain with header configuration
4. Enable HSTS if HTTPS-only

4. Risk Assessment Matrix

Severity Distribution

CRITICAL	3
HIGH	5
MEDIUM	8
LOW	4

Business Risk Summary

- Revenue Loss: VIP bypass and client-side premium control enable free access to paid content
- Data Breach: PII collection without encryption exposes users to privacy violations
- Compliance: GDPR/privacy violations from undisclosed tracking
- Reputation: Security failures damage user trust
- Operational: Admin email exposure and session persistence create attack surface

5. Remediation Roadmap

Immediate Actions (0-7 days)

- SMC-001: Remove all client-side VIP codes
- SMC-002: Move premium control to Firebase server-side
- SMC-003: Implement server-side payment verification
- SMC-005: Replace email-based admin with Custom Claims

Short-term (1-4 weeks)

- SMC-004: Configure Firebase API key restrictions
- SMC-006: Tighten Firebase security rules
- SMC-007: Implement consent and data minimization
- SMC-008: Add Content Security Policy
- SMC-011: Move question banks server-side

Medium-term (1-3 months)

- SMC-009: Encrypt sensitive localStorage
- SMC-012: Implement session timeout
- SMC-015: Add fingerprinting consent
- SMC-016: Add rate limiting
- SMC-020: Add security headers

Long-term (3-6 months)

- Implement comprehensive monitoring and alerting
- Regular security audits and penetration testing
- Security training for development team
- Bug bounty program consideration

6. Conclusion

The SMC Computer Supervisor Exam Preparation website has critical security vulnerabilities that directly impact revenue protection, user privacy, and data integrity. The most severe issues relate to premium access control being enforced client-side rather than server-side, allowing trivial bypass of paid content.

Immediate remediation of the three Critical findings (SMC-001, SMC-002, SMC-003) is essential to protect revenue and maintain the integrity of the subscription model. The High findings related to Firebase configuration and PII handling should be addressed within the first month.

Recommendation: Engage a security professional to implement server-side access control via Firebase Cloud Functions and Custom Claims, conduct a follow-up penetration test after remediation, and establish ongoing security monitoring.

END OF REPORT